PRACTICAL – 3

Implementation of Max-Heap Sort Algorithm

Objective:

*The objective of this practical is to **implement the Max Heap Sort algorithm** using Python. The algorithm arranges the elements into a **Max Heap** and repeatedly extracts the **maximum element** to sort the array in ascending order.*

Max Heap Sort

Algorithm:

Start
For i = n/2 - 1 to 0
Apply MaxHeapify(A, n, i)
End For
For i = n - 1 to 1
Swap A[0] and A[i]
Apply MaxHeapify(A, i, 0)
End For
Stop

Heap Sort

Algorithm:

Start
Build a Max Heap from the given array A.
For i = n - 1 to 1
Swap A[0] and A[i]
Apply MaxHeapify(A, i, 0)
End For
Stop

Python Code:

```
# Max Heapify Function
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    # Check left child
    if left < n and arr[left] > arr[largest]:
        largest = left

    # Check right child
    if right < n and arr[right] > arr[largest]:
        largest = right

    # If largest is not root
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]

        # Heapify the affected subtree
        heapify(arr, n, largest)

# Heap Sort Function
def heap_sort(arr):
    n = len(arr)

    # Build Max Heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # Extract elements from heap
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]

        # Heapify the reduced heap
        heapify(arr, i, 0)
```

```
# Take input from the user
n = int(input("Enter the number of elements: "))

arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Display original array
print("\nOriginal Array:", arr)

# Perform Heap Sort
heap_sort(arr)

# Display sorted array
print("Sorted Array:", arr)

# Enter your EN.No
print("EN.No:92460118708")
```

OUTPUT:

```
PS C:\Users\penta\Downloads\DAA-practical-3-main>  & 'C
s\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\deb
HeapSort.py'
Enter the number of elements: 4
Enter 4 elements:
34
45
67
78
Sorted array: 34 45 67 78

Enrollment Number: 92460118708
```

Time Complexity:

Case	*Time Complexity*	*Description*
Best Case	*O(n log n)*	*The heap construction and extraction operations take O(n log n).*
Average Case	*O(n log n)*	*Each extraction requires heapification, giving O(n log n).*
Worst Case	*O(n log n)*	*Heap Sort still performs in O(n log n).*

Space Complexity:

Space Complexity	*Description*
O(log n)	*The recursive heapify() function uses the recursion stack.*